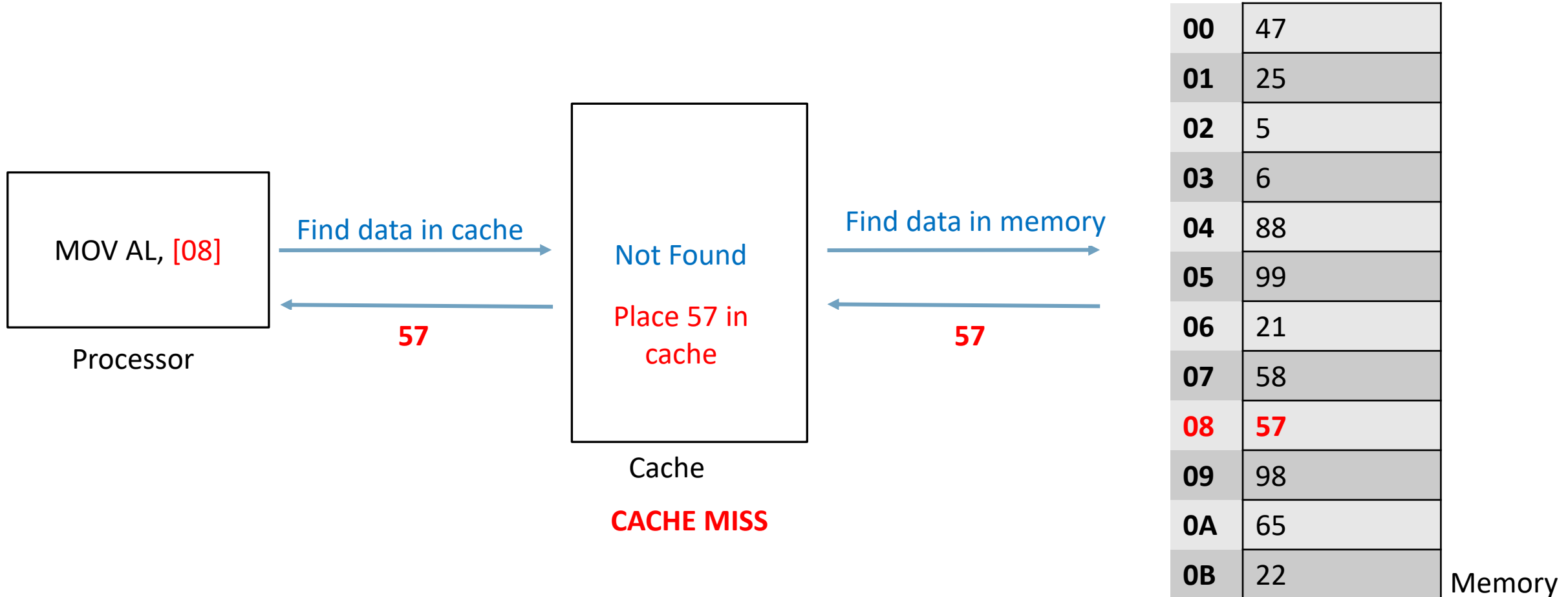


Cache

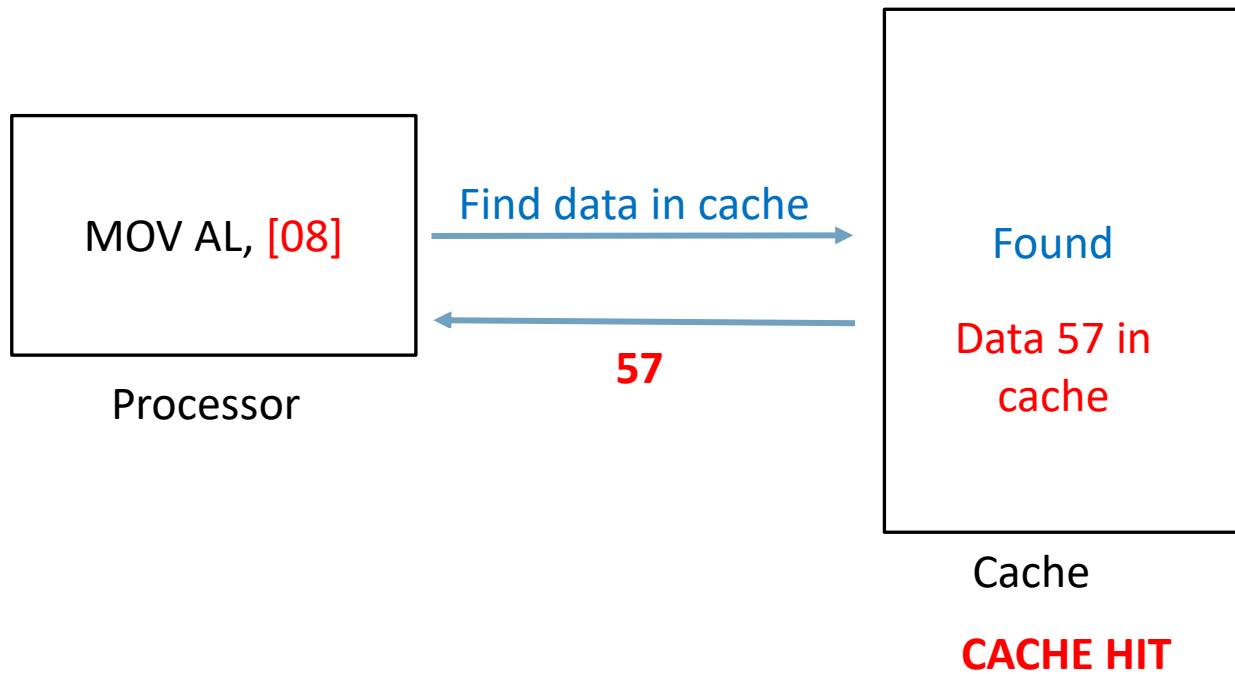
COMPILED BY:

SAROSH SHAHID

Cache Miss



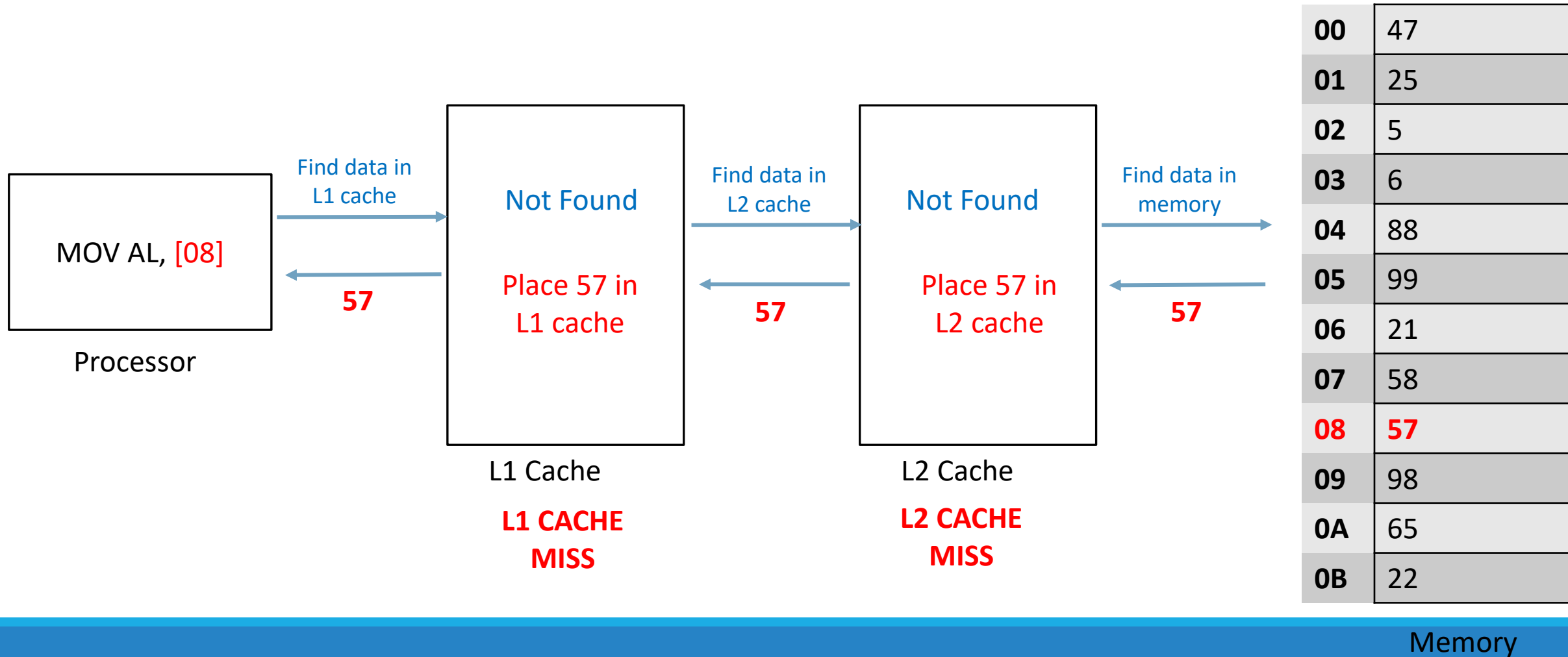
Cache Hit



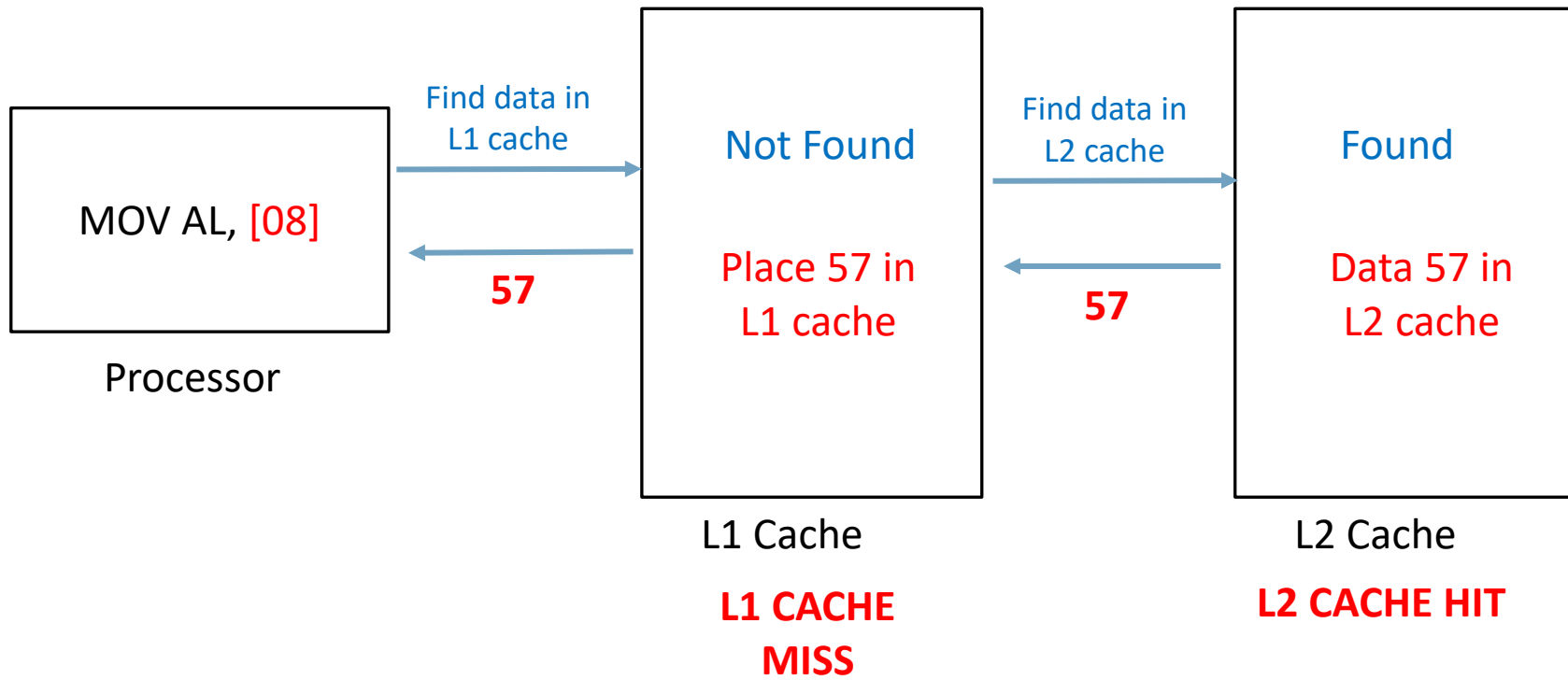
00	47
01	25
02	5
03	6
04	88
05	99
06	21
07	58
08	57
09	98
0A	65
0B	22

Memory

Cache Miss



Cache Miss



00	47
01	25
02	5
03	6
04	88
05	99
06	21
07	58
08	57
09	98
0A	65
0B	22

Memory

Locality

Temporal Locality:

- Locality in time
- If data used recently, likely to use it again soon
- **How to exploit:** keep recently accessed data in higher levels of memory hierarchy

Spatial Locality:

- Locality in space
- If data used recently, likely to use nearby data soon
- **How to exploit:** when access data, bring nearby data into higher levels of memory hierarchy too

Locality

```
for (int i = 0; i < 10000; i++)  
    a = A[i] + a
```

Temporal Locality:

*If data used recently, likely to use it again soon “Variables **i** and **a**”*

Spatial Locality:

*If data used recently, likely to use nearby data soon “Array **A**”*

What data is held in cache?

Ideally, cache anticipates needed data and puts it in cache

But impossible to predict future

Use past to predict future – temporal and spatial locality:

- **Temporal locality:** copy newly accessed data into cache
- **Spatial locality:** copy neighboring data into cache too

Cache Performance Analysis

$$\text{Miss Rate} = \frac{\text{Number of misses}}{\text{Number of total memory Accesses}} = 1 - \text{Hit Rate}$$

$$\text{Hit Rate} = \frac{\text{Number of hits}}{\text{Number of total memory Accesses}} = 1 - \text{Miss Rate}$$

Cache Performance Analysis

Example

Suppose a program has 2000 data access instructions (load or stores), and 1250 of these requested data values are found in the cache. The other 750 data values are supplied to the processor by main memory or disk memory. What are the miss and hit rates for the cache?

$$\text{Miss Rate} = \frac{\text{Number of misses}}{\text{Number of total memory Accesses}} = 1 - \text{Hit Rate}$$

$$\text{Hit Rate} = \frac{\text{Number of hits}}{\text{Number of total memory Accesses}} = 1 - \text{Miss Rate}$$

Solution

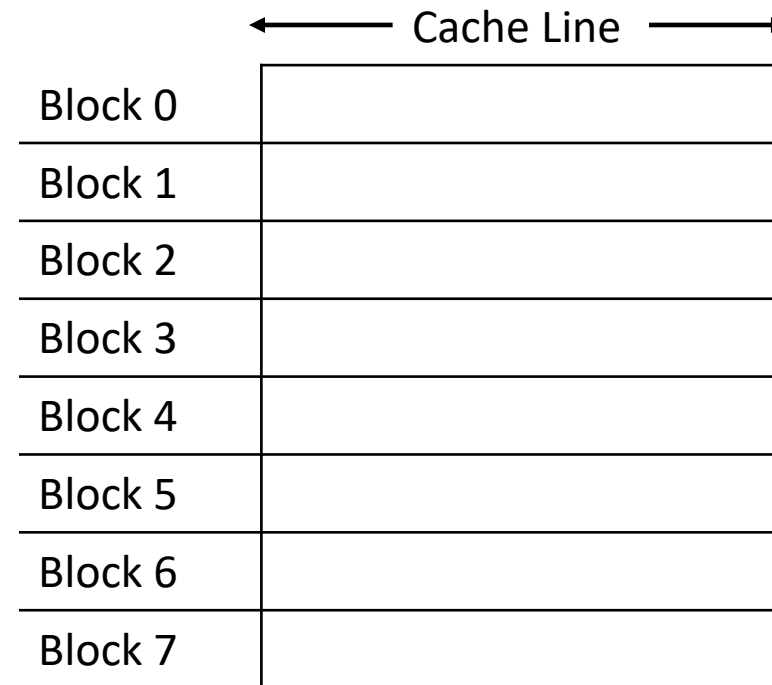
$$\text{Miss Rate} = \frac{\text{Number of misses}}{\text{Number of total memory Accesses}} = \frac{750}{2000} = 0.375 = 37.5\%$$

$$\text{Hit Rate} = 1 - \text{Miss Rate} = 1 - 0.375 = 62.5\%$$

Cache Terminologies

- Cache Capacity C = Total number of bytes that can be stored in the cache
- Data is transferred between memory and cache in blocks of fixed size called cache lines or cache blocks.

b = Block size

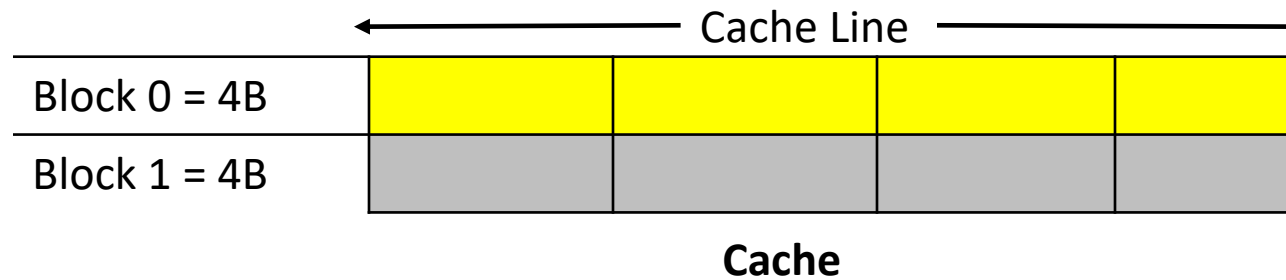


Cache

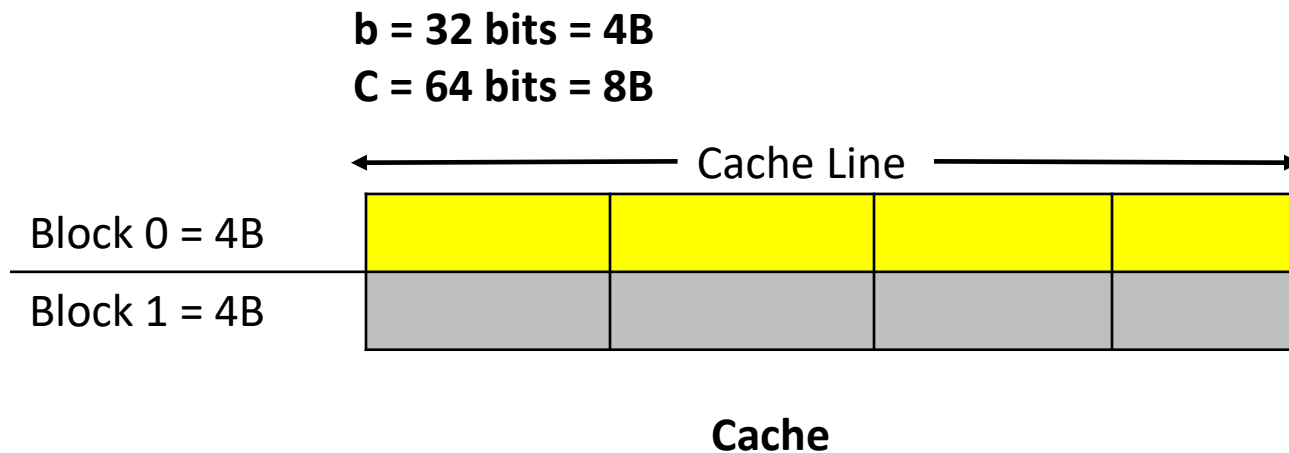
Cache Terminologies

b = 32 bits = 4B

C = 64 bits = 8B



Block Size



One block of cache can hold 4Bytes so memory is also logically divided into equal chunks of 4 bytes and all 4 bytes are brought in the cache at once.

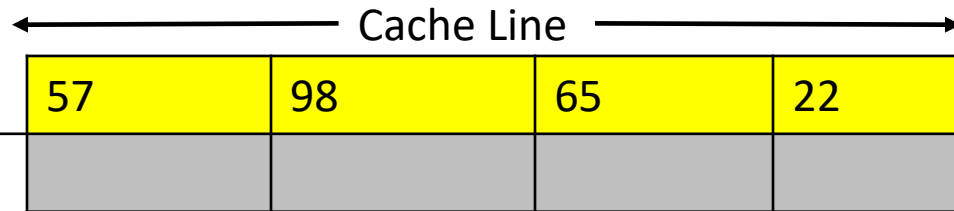
Block 0	00	47
	01	25
	02	5
	03	6
Block 1	04	88
	05	99
	06	21
	07	58
Block 2	08	57
	09	98
	0A	65
	0B	22
Block 3	0C	66
	0D	8
	0E	55
	0F	1

Memory

Block Size

b = 32 bits = 4B

C = 64 bits = 8B



Cache

MOV AL, [09]

Instead of bringing only [09] will bring the whole block in the cache

Block size = 4B so will bring 4Bytes and as the address [09] is in **“Block 2”** and it maps to the **“yellow colored”** block of the cache... **(Spatial Locality)**

Block 0	00	47
	01	25
	02	5
	03	6
Block 1	04	88
	05	99
	06	21
	07	58
Block 2	08	57
	09	98
	0A	65
	0B	22
Block 3	0C	66
	0D	8
	0E	55
	0F	1

Offset Bits

B = 32 bits = 4B

C = 64 bits = 8B

***Offset bits* = $\log_2 b = \log_2 4 = 2$**

	Cache Line			
Offset	00	01	10	11
Block 0 = 4B	57	98	65	22
Block 1 = 4B				

Cache

MOV AL, [09]

Offset bits are used to identify the data

[09] is at offset **"01b or 1"**

[0A] is at offset **"10b or 2"** and so on

Block 0	00	47
	01	25
	02	5
	03	6
Block 1	04	88
	05	99
	06	21
	07	58
Block 2	08	57
	09	98
	0A	65
	0B	22
Block 3	0C	66
	0D	8
	0E	55
	0F	1

Cache Terminology

Capacity (C):

- number of data bytes in cache

Block size (b):

- bytes of data brought into cache at once

Number of blocks ($B = C/b$):

- number of blocks in cache: $B = C/b$

Degree of associativity (N):

- number of blocks in a set

Number of sets ($S = B/N$):

- each memory address maps to exactly one cache set

Cache Types

Direct Mapped Cache

Set # 0	
Set # 1	
Set # 2	
Set # 3	
Set # 4	
Set # 5	
Set # 6	
Set # 7	

One-way Associative Cache

1 block per set

N-way Associative Cache

Set # 0		Set # 0	
Set # 1		Set # 1	
Set # 2		Set # 2	
Set # 3		Set # 3	

Two-way Associative Cache

Four-way Associative Cache

N blocks per set

Fully Associative Cache

Set # 0	

All cache blocks in one set

Design Cache

b = 4B

C = 64B

N = 1 (Direct Mapped)

$$B = C/b = 16$$

$$S = B/N = 16/1 = 16$$

$$\text{Offset bits} = \log_2 b = \log_2 4 = 2$$

$$\text{Set/index bits} = \log_2 S = \log_2 16 = 4$$

	Offset	00	01	10	11
Sets					
0000					
0001					
0010					
0011					
0100					
0101					
.....					
1101					
1110					
1111					

Design Cache

b = 32B

C = 128B

N = 1 (Direct Mapped)

$$B = C/b = 128/32 = 4$$

$$S = B/N = 4/1 = 4$$

$$\text{Offset bits} = \log_2 b = \log_2 32 = 5$$

$$\text{Set/index bits} = \log_2 S = \log_2 4 = 2$$

	Offset	00000	00001	00010		11110	11111
Sets							
00							
01							
10							
11							

Design Cache

b = 4B

C = 32B

N = 4 (4-way associative)

$$B = C/b = 32/4 = 8$$

$$S = B/N = 8/4 = 2$$

$$\text{Offset bits} = \log_2 b = \log_2 4 = 2$$

$$\text{Set/index bits} = \log_2 S = \log_2 2 = 1$$

Offset		00	01	10	11
Sets					
0					
1					

Direct Memory

[illegible]

Design Cache

b = 8B

Full Associative Cache

Single set with 4-ways

$$C = 8B * 4 \text{ (ways)} = 32B$$

$$B = C/b = 32/8 = 4$$

$$S = B/N = 4/4 = 1$$

$$\text{Offset bits} = \log_2 b = \log_2 8 = 3$$

$$\text{Set/index bits} = \log_2 S = \log_2 1 = 0$$

Offset	000	001	010	011	100	101	110	111
Sets								
0								

Cache Terminologies

Tag Bits - A unique identifier for a group of data. Because different regions of memory may be mapped into a single block, the tag is used to differentiate between them.

We need to add tags to the cache, which supply the rest of the address bits to let us distinguish between different memory locations that map to the same cache block.

Valid bit (V)- A bit of information that indicates whether the data in a block is valid (1) or not (0).

1. At the very start, the cache is empty and does not contain valid data.
2. We should account for this by adding a valid bit for each cache block.
3. When the system is initialized, all the valid bits are set to 0. When data is loaded into a particular cache block, the corresponding valid bit is set to 1.

0x014 $\xrightarrow[\text{binary}]{\text{Convert to}}$ 0000 0001 0100

Tag Bits Set bits Offset bits

Example 01

b = 4B
C = 32B
N = 1 (Direct Mapped)
Addresses are 12 bits

$B = C/b = 8$
 $S = B/N = 8/1 = 8$
Offset bits = $\log_2 b = \log_2 4 = 2$
Set/index bits = $\log_2 S = \log_2 8 = 3$
Tag bits = *Address bits* – *Offset bits* – *set bits*
 = $12 - 2 - 3 = 7$

Processor Memory Access Sequence

1. 0x014
2. 0xFF0
3. 0x0F7
4. 0xC03
5. 0x0F3

Sets	V bit	Tags	00	01	10	11
000	0					
001	0					
010	0					
011	0					
100	0					
101	1	0000000	00	00	00	14
110	0					
111	0					

Cache

000	00000000
004	00000004
008	00000008
00C	0000000C
010	00000010
014	00000014
018	00000018
01C	0000001C
020	00000020
.....	
FF4	00000FF4
FF8	00000FF8
FFC	00000FFC

Memory

Cache Miss = 1 Cache Hit = 0

Example 01

b = 4B
C = 32B
N = 1 (Direct Mapped)
Addresses are 12 bits

$B = C/b = 8$
 $S = B/N = 8/1 = 8$
 $Offset\ bits = \log_2 b = \log_2 4 = 2$
 $Set/index\ bits = \log_2 S = \log_2 8 = 3$
 $Tag\ bits = Address\ bits - Offset\ bits - set\ bits$
 $= 12 - 2 - 3 = 7$

Processor Memory Access Sequence

1. 0x014
2. 0xFF0
3. 0x0F7
4. 0xC03
5. 0x0F3

Sets	V bit	Tags	00	01	10	11
000	0					
001	0					
010	0					
011	0					
100	1	1111111	00	00	0F	F0
101	1	0000000	00	00	00	14
110	0					
111	0					

Cache

0xFF0 $\xrightarrow{\text{Convert to binary}}$ 1111 1111 0000
 Tag Bits Set bits Offset bits

000	00000000
004	00000004
008	00000008
00C	0000000C
010	00000010
014	00000014
018	00000018
01C	0000001C
020	00000020
.....	
FF4	00000FF4
FF8	00000FF8
FFC	00000FFC

Memory

Cache Miss = 2 Cache Hit = 0

Example 01

b = 4B
C = 32B
N = 1 (Direct Mapped)
Addresses are 12 bits

$B = C/b = 8$
 $S = B/N = 8/1 = 8$
 $Offset\ bits = \log_2 b = \log_2 4 = 2$
 $Set/index\ bits = \log_2 S = \log_2 8 = 3$
 $Tag\ bits = Address\ bits - Offset\ bits - set\ bits$
 $= 12 - 2 - 3 = 7$

Processor Memory Access Sequence

1. 0x014
2. 0xFF0
3. 0x0F7
4. 0xC03
5. 0x0F3

0x0F7 $\xrightarrow[\text{binary}]{\text{Convert to}}$ **0000 1111 0111**
 Tag Bits Set bits Offset bits

Sets	V bit	Tags	00	01	10	11
000	0					
001	0					
010	0					
011	0					
100	1	1111111	00	00	0F	F0
101	1	0000111	00	00	00	14
110	0					
111	0					

Cache

000	00000000
004	00000004
008	00000008
00C	0000000C
010	00000010
014	00000014
018	00000018
01C	0000001C
020	00000020
.....	
FF4	00000FF4
FF8	00000FF8
FFC	00000FFC

Memory

Data already present in the set "101". To check whether it's the same data we need "compare the tag bits". As $0000000 \neq 0000111$ It means we have to replace the data.

Cache Miss = 3 **Cache Hit = 0**

0xC03 $\xrightarrow{\text{Convert to binary}}$ 1100 0000 0011

Tag Bits Set bits Offset bits

Example 01

$b = 4B$
 $C = 32B$
 $N = 1$ (Direct Mapped)
 Addresses are 12 bits

$B = C/b = 8$
 $S = B/N = 8/1 = 8$
 $\text{Offset bits} = \log_2 b = \log_2 4 = 2$
 $\text{Set/index bits} = \log_2 S = \log_2 8 = 3$
 $\text{Tag bits} = \text{Address bits} - \text{Offset bits} - \text{set bits}$
 $= 12 - 2 - 3 = 7$

Processor Memory Access Sequence

1. 0x014
2. 0xFF0
3. 0x0F7
4. 0xC03
5. 0x0F3

Sets	V bit	Tags	00	01	10	11
000	0	1100000	00	00	0C	03
001	0					
010	0					
011	0					
100	1	1111111	00	00	0F	F0
101	1	0000111	00	00	00	F7
110	0					
111	0					

Cache

000	00000000
004	00000004
008	00000008
00C	0000000C
010	00000010
014	00000014
018	00000018
01C	0000001C
020	00000020
.....	
FF4	00000FF4
FF8	00000FF8
FFC	00000FFC

Memory

Cache Miss = 4 Cache Hit = 0

0x0F3 $\xrightarrow[\text{binary}]{\text{Convert to}}$ 0000 1111 0011

Tag Bits Set bits Offset bits

Example 01

b = 4B
C = 32B
N = 1 (Direct Mapped)
Addresses are 12 bits

$B = C/b = 8$
 $S = B/N = 8/1 = 8$
Offset bits = $\log_2 b = \log_2 4 = 2$
Set/index bits = $\log_2 S = \log_2 8 = 3$
Tag bits = *Address bits* – *Offset bits* – *set bits*
 = $12 - 2 - 3 = 7$

Processor Memory Access Sequence

1. 0x014
2. 0xFF0
3. 0x0F7
4. 0xC03
5. **0x0F3**

Sets	V bit	Tags	00	01	10	11
000	1	1100000	00	00	0C	03
001	0					
010	0					
011	0					
100	1	0000111	00	00	00	F7
101	1	0000000	00	00	00	14
110	0					
111	0					

Cache

000	00000000
004	00000004
008	00000008
00C	0000000C
010	00000010
014	00000014
018	00000018
01C	0000001C
020	00000020
.....	
FF4	00000FF4
FF8	00000FF8
FFC	00000FFC

Memory

Data already present in the set “100”. To check whether it’s the same data we need “compare the tag bits”. As 0000111 = 0000111 It means we found the data and it’s a **CACHE HIT**.

Cache Miss = 4 Cache Hit = 1